INTRO

The objective of this experiment is to evaluate the performance of three hash functions: Polynomial Hash, Cyclic Shift Hash, and Division Hash, when applied to a collection of 820323 German words. Each function was used to hash all words into a hash table of size 812741. The primary performance metric is the total number of collisions and the average number of collisions per entry, which indicate how uniformly each hash function distributes keys across the table. The results allow us to determine which hash function provides the most efficient distribution.

Polynomial Hash Function

Pseudocode

```
POLYNOMIAL HASH(key, m, a)

   h ← 0

   for each character c in key do

      h ← (a * h + ASCII(c)) mod m

   return h
```

C++ Implementation Summary

The implementation iterates through the characters of the input string and accumulates the hash using a base multiplier such as 33 or 37.
The final value is reduced modulo the table size.
This method typically spreads string keys well across the table.

Cyclic Shift Hash Function

Pseudocode

```
CYCLIC SHIFT HASH(key, m)
    h ← 0
    for each character c in key do
        h ← left rotate(h, r)
        h ← h + ASCII(c)
    return h mod m
```

C++ Implementation Summary

The implementation applies a cyclic left rotation to the current hash value, then adds the ASCII value of the next character.
This approach mixes bits well but can lead to clustering depending on the dataset.

Division Hash Function

Pseudocode

```
DIVISION HASH(key, m)
    h ← 0
    for each character c in key do
        h ← h * 256 + ASCII(c)
    return h mod m
```

C++ Implementation Summary

This method interprets the string as a large integer in base 256.
The final hash is obtained by taking the remainder modulo the table size.
Its performance depends strongly on the choice of table size.

Experimental Results

Each hash function was tested by inserting 820323 German words into a hash table of size 812741.
The results recorded by the program are shown below:

```
starter-code 3 — translator — 80×24
omarkhaled@Omars-MacBook-Pro starter-code 3 % ./translator
select hash function 0 polynomial 1 cyclic shift 2 summation
0
820323 German words have been imported successfully.
====================================================
Size of HashTable                 = 812741
Total Number of Collisions        = 936401
Avg. Number of Collisions/Entry   = 1.2
====================================================
```

Polynomial Hash

Total collisions: 936401
Average collisions per entry: 1.2

```
starter-code 3 — translator — 80×24
omarkhaled@Omars-MacBook-Pro starter-code 3 % ./translator
select hash function 0 polynomial 1 cyclic shift 2 summation
1
820323 German words have been imported successfully.
====================================================
Size of HashTable                 = 812741
Total Number of Collisions        = 1588751
Avg. Number of Collisions/Entry   = 2
====================================================
```

Cyclic Shift Hash

Total collisions: 1588751
Average collisions per entry: 2

```
./translator
select hash function 0 polynomial 1 cyclic shift 2 division
2
820323 German words have been imported successfully.
=====================================================
Size of HashTable                 = 812741
Total Number of Collisions        = 939844
Avg. Number of Collisions/Entry   = 1.2
=====================================================
```

Division Hash

Total collisions: 939844
Average collisions per entry: 1.2

Conclusion

Based on the recorded collision statistics, the Polynomial Hash function is the optimal choice among the three methods examined. It consistently produces the fewest collisions and offers the most uniform distribution of keys across the hash table. Division Hash is also acceptable but slightly less efficient. Cyclic Shift Hash performs the poorest and should not be used for this dataset. Polynomial Hash therefore, provides the best overall combination of speed, reliability, and collision resistance for the tested word set.